

Class-based Modeling

Tahir Farooq

Outline

- What is a class diagram?
- Identifying classes
- Elements of a UML Class Diagram
 - Associations
 - Generalization
 - Dependencies

Essential Elements of a UML Class Diagram

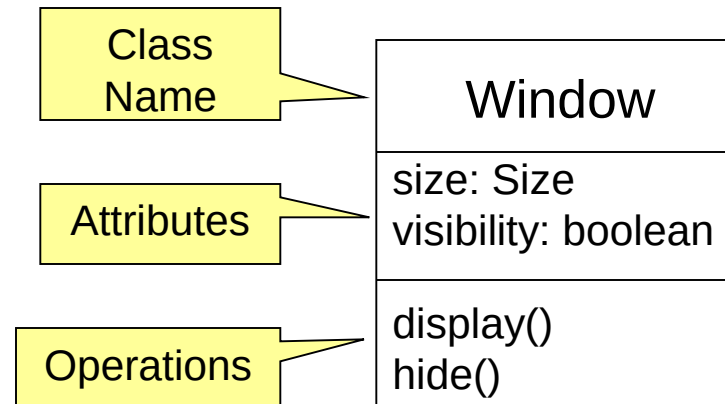
- Class
- Attributes
- Operations
- Relationships
 - Associations
 - Generalization
 - Dependency
 - Realization
- Constraint Rules and Notes

What is a Class Diagram?

- A **class diagram** describes the types of objects in the system and the various kinds of **static relationships** that exist among them.
 - A graphical representation of a static view on declarative static elements.
- A central modeling technique that runs through nearly all object-oriented methods.
- The richest notation in UML.

Classes

- A class is the description of a set of objects having similar attributes, operations, relationships and behavior.



Identifying Analysis Classes

- 1) Perform a grammatical parse of the problem statement or use cases
- 2) Classes are determined by underlining each noun or noun clause
- 3) A class required to implement a solution is part of the solution space
- 4) A class necessary only to describe a solution is part of the problem space
- 5) List the potential class names in a table and "classify" each class according to some taxonomy and class selection characteristics
- 6) A potential class should satisfy nearly all (or all) of the selection characteristics to be considered a legitimate problem domain class

Potential classes	General classification	Selection Characteristics

(More on next slide)

Grammatical Parse

- The *SafeHome* security function enables the homeowner to configure the security system when it is installed, monitors all sensors connected to the security system, and interacts with the homeowner through the Internet, a PC, or a control panel.
- During installation, the *SafeHome* PC is used to program and configure the system. Each sensor is assigned a number and type, a master password is programmed for arming and disarming the system, and telephone number(s) are input for dialing when a sensor event occurs.
- When a sensor event is recognized, the software invokes an audible alarm attached to the system. After a delay time that is specified by the homeowner during system configuration activities, the software *dials* a telephone number of a monitoring service, provides information about the location, reporting the nature of the event that has been detected. The telephone number will be redialed every 20 seconds until a telephone connection is obtained.
- The homeowner receives security information via a control panel, the PC, or a browser, collectively called an interface. The interface displays prompting messages and system status information on the control panel, the PC, or the browser window. Homeowner interaction takes the following form...

Grammatical Parse

- The *SafeHome* **security function** enables the **homeowner** to configure the **security system** when it is installed, monitors all **sensors** connected to the security system, and interacts with the homeowner through the **Internet**, a **PC**, or a **control panel**.
- During **installation**, the *SafeHome* PC is used to program and configure the system. Each sensor is assigned a **number** and **type**, a **master password** is programmed for arming and disarming the system, and **telephone number(s)** are input for dialing when a **sensor event** occurs.
- When a sensor event is recognized, the software invokes an audible **alarm** attached to the system. After a **delay time** that is specified by the homeowner during system configuration activities, the software *dials* a telephone number of a **monitoring service**, provides information about the **location**, reporting the nature of the event that has been detected. The telephone number will be redialed every 20 seconds until a **telephone connection** is obtained.
- The homeowner receives **security information** via a control panel, the PC, or a browser, collectively called an **interface**. The interface displays prompting **messages** and system **status information** on the control panel, the PC, or the browser window. Homeowner interaction takes the following form...

Identifying Analysis Classes (continued)

- General classifications for a potential class
 - External entity (e.g., another system, a device, a person)
 - Thing (e.g., report, screen display)
 - Occurrence or event (e.g., movement, completion)
 - Role (e.g., manager, engineer, salesperson)
 - Organizational unit (e.g., division, group, team)
 - Place (e.g., manufacturing floor, loading dock)
 - Structure (e.g., sensor, vehicle, computer)

(More on next slide)

Class Selection Criteria

1. Retained information
2. Needed services
3. Multiple attributes
4. Common attributes
5. Common operations
6. Essential requirements

Identifying Classes

Potential class	Classification	Accept / Reject
homeowner	role; external entity	reject: 1, 2 fail
sensor	external entity	accept
control panel	external entity	accept
installation	occurrence	reject
(security) system	thing	accept
number, type	not objects, attributes	reject: 3 fails
master password	thing	reject: 3 fails
telephone number	thing	reject: 3 fails
sensor event	occurrence	accept
audible alarm	external entity	accept: 1 fails
monitoring service	organizational unit;	reject: 1, 2 fail

Defining Attributes of a Class

- Attributes of a class are those nouns from the grammatical parse that reasonably belong to a class
- Attributes hold the values that describe the current properties or state of a class
- An attribute may also appear initially as a potential class that is later rejected because of the class selection criteria
- In identifying attributes, the following question should be answered
 - What data items (composite and/or elementary) will fully define a specific class in the context of the problem at hand?
- Usually an item is not an attribute if more than one of them is to be associated with a class

Defining Operations of a Class

- Operations define the behavior of an object
- Four categories of operations
 - Operations that manipulate data in some way to change the state of an object (e.g., add, delete, modify)
 - Operations that perform a computation setName(name: String), addStudent(student: Student)
 - Operations that inquire about the current state of an object calculateTotalPrice()
 - Operations that monitor an object for the occurrence of a controlling event. getBalance() isAvailable()
getName()
- An operation has knowledge about the state of a class and the nature of its associations
- The action performed by an operation is based on the current values of the attributes of a class
- Using a grammatical parse again, **circle the verbs**; then select the verbs that relate to the problem domain classes that were previously identified

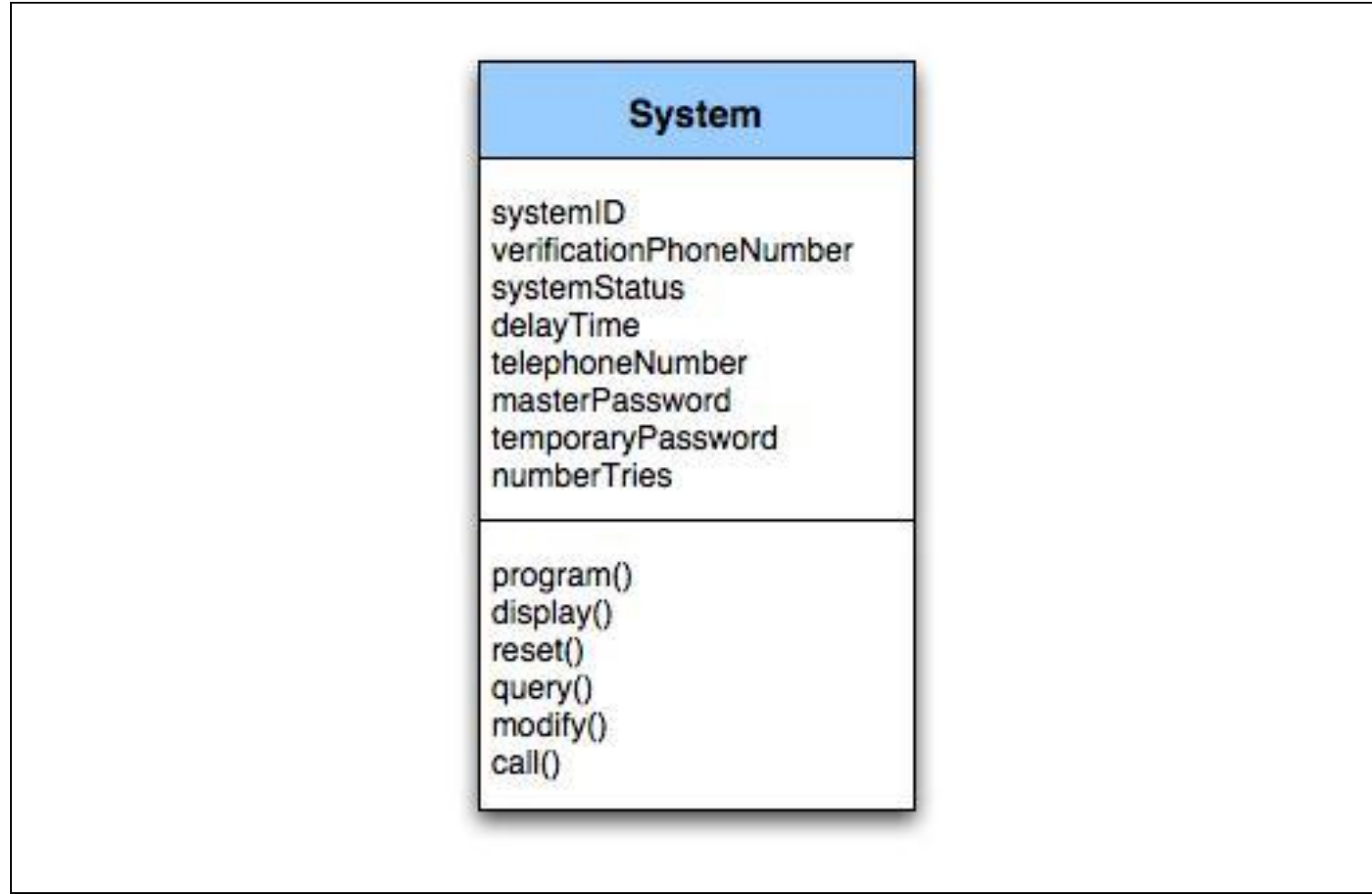
Identifying operations

- The *SafeHome* security function enables the homeowner to configure the security system when it is installed, monitors all sensors connected to the security system, and interacts with the homeowner through the Internet, a PC, or a control panel.
- During installation, the *SafeHome* PC is used to program and configure the system. Each sensor is assigned a number and type, a master password is programmed for arming and disarming the system, and telephone number(s) are input for dialing when a sensor event occurs.
- When a sensor event is recognized, the software invokes an audible alarm attached to the system. After a delay time that is specified by the homeowner during system configuration activities, the software *dials* a telephone number of a monitoring service, provides information about the location, reporting the nature of the event that has been detected. The telephone number will be redialed every 20 seconds until a telephone connection is obtained.
- The homeowner receives security information via a control panel, the PC, or a browser, collectively called an interface. The interface displays prompting messages and system status information on the control panel, the PC, or the browser window. Homeowner interaction takes the following form...

Identifying Operations

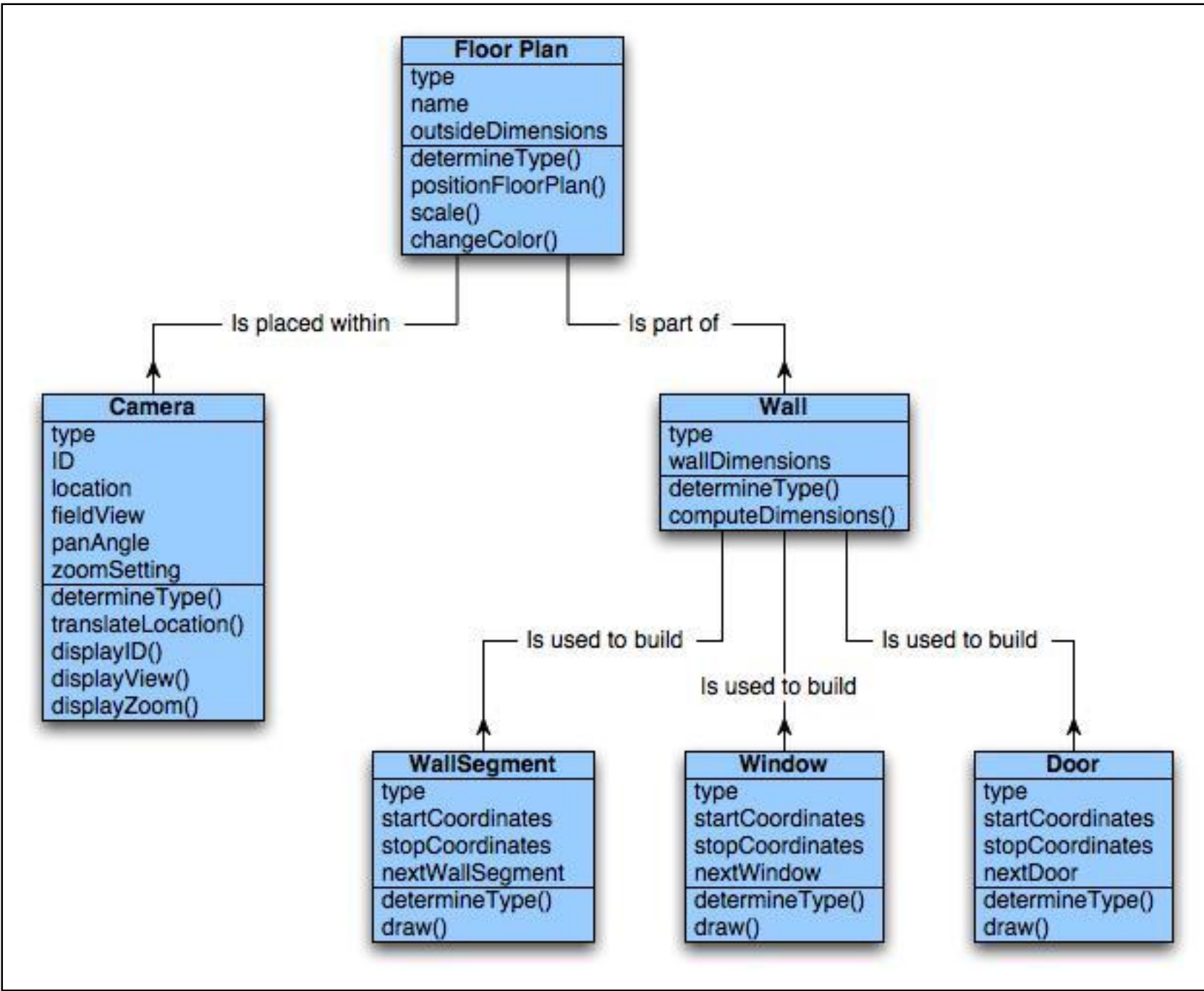
- The *SafeHome* security function enables the homeowner to configure the security system when it is installed, monitors all sensors connected to the security system, and interacts with the homeowner through the Internet, a PC, or a control panel.
- During installation, the *SafeHome* PC is used to program and configure the system. Each sensor is assigned a number and type, a master password is programmed for arming and disarming the system, and telephone number(s) are input for dialing when a sensor event occurs.
- When a sensor event is recognized, the software invokes an audible alarm attached to the system. After a delay time that is specified by the homeowner during system configuration activities, the software *dials* a telephone number of a monitoring service, provides information about the location, reporting the nature of the event that has been detected. The telephone number will be redialed every 20 seconds until a telephone connection is obtained.
- The homeowner receives security information via a control panel, the PC, or a browser, collectively called an interface. The interface displays prompting messages and system status information on the control panel, the PC, or the browser window. Homeowner interaction takes the following form...

Class Diagram



Class diagram for the system class

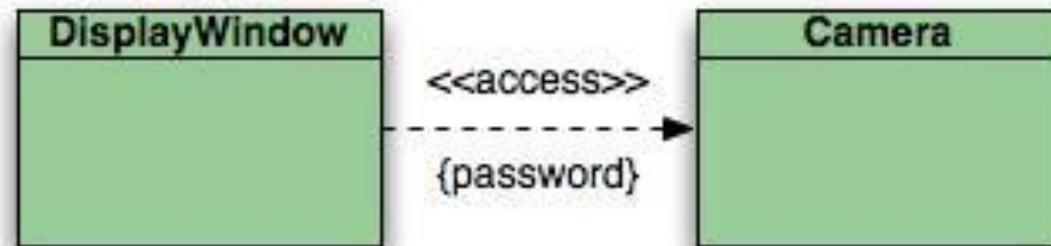
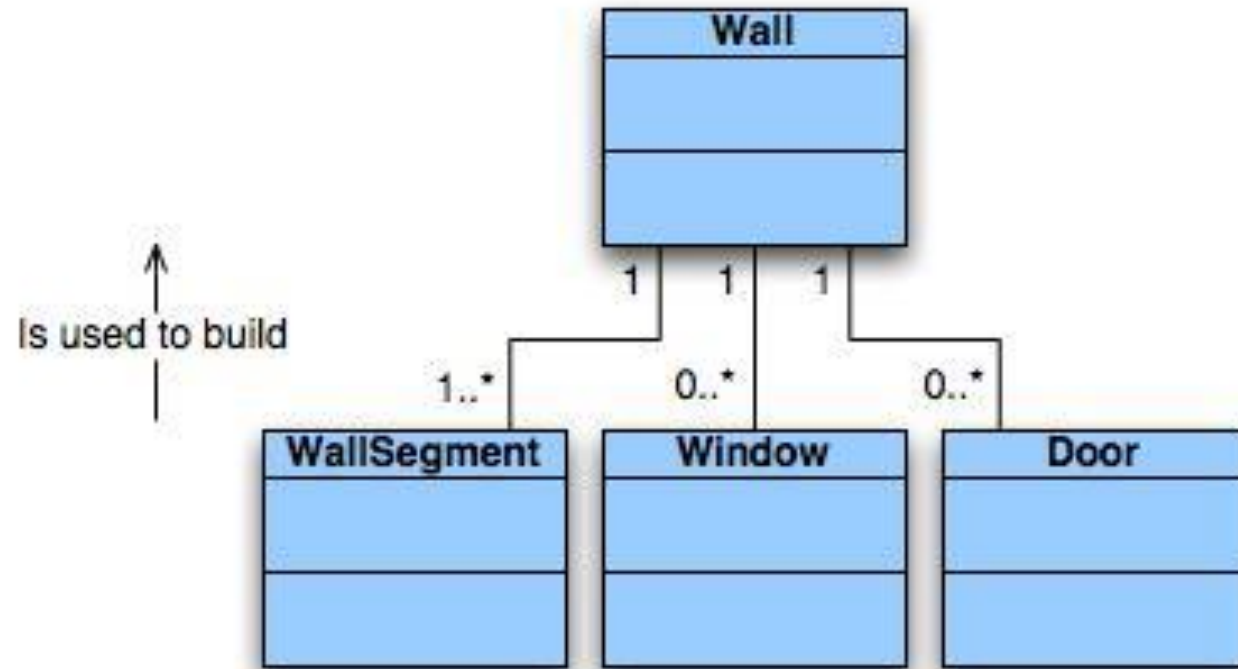
Class Diagram



Class diagram for FloorPlan

Class Diagrams

Top: Multiplicity
Bottom: Dependencies



Associations

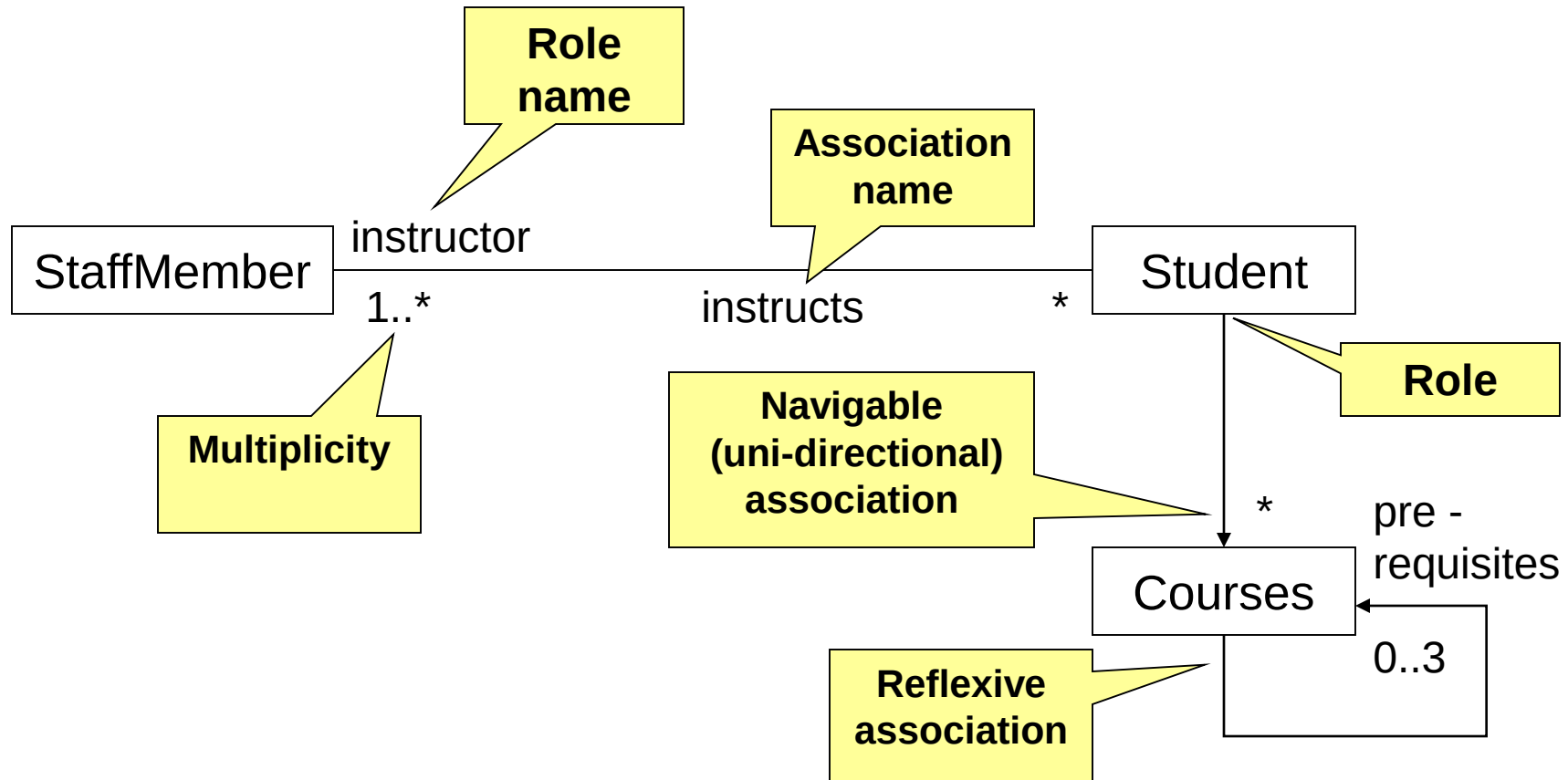
- A semantic relationship between two or more classes that specifies connections among their instances.
- A structural relationship, specifying that objects of one class are connected to objects of a second (possibly the same) class.
- Example: “An Employee works for a Company”



Associations (cont.)

- An association between two classes indicates that objects at one end of an association “recognize” objects at the other end and may send messages to them.
 - This property will help us discover less trivial associations using interaction diagrams.

Associations (cont.)



Associations (cont.)

- To clarify its meaning, an association may be named.
 - The name is represented as a label placed midway along the association line.
 - Usually a verb or a verb phrase.
- A **role** is an end of an association where it connects to a class.
 - May be named to indicate the role played by the class attached to the end of the association path.
 - Usually a noun or noun phrase
 - Mandatory for reflexive associations

Associations (cont.)

- Multiplicity
 - The number of instances of the class, next to which the multiplicity expression appears, that are referenced by a **single** instance of the class that is at the other end of the association path.
 - Indicates whether or not an association is mandatory.
 - Provides a lower and upper bound on the number of instances.

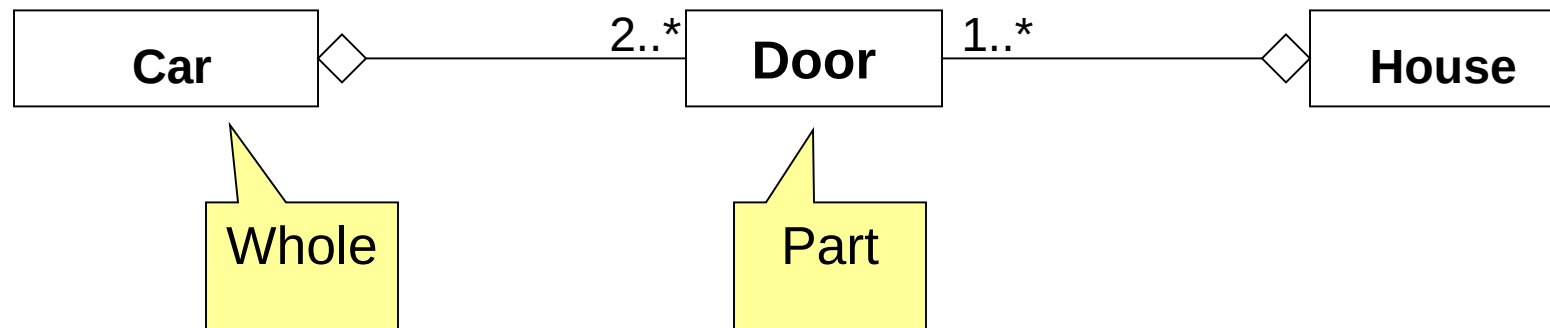
Associations (cont.)

– Multiplicity Indicators

Exactly one	1
Zero or more (unlimited)	* (0..*)
One or more	1..*
Zero or one (optional association)	0..1
Specified range	2..4
Multiple, disjoint ranges	2, 4..6, 8

Aggregation

- A special form of association that models a whole-part relationship between an aggregate (the whole) and its parts.
 - Models a “is a part-part of” relationship.

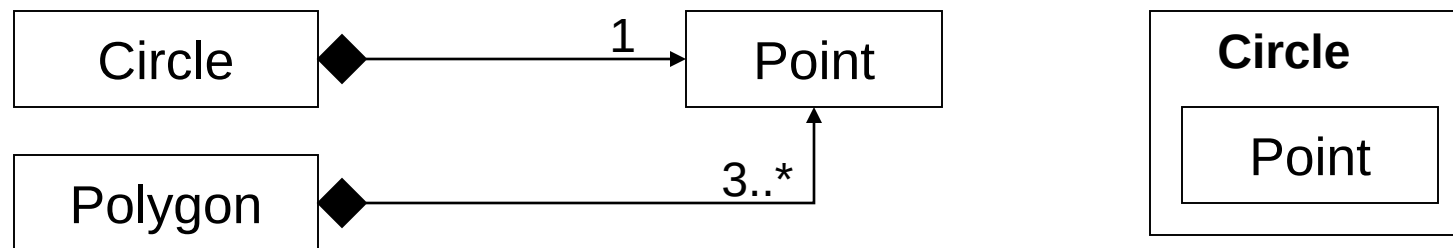


Aggregation (cont.)

- Aggregation tests:
 - Is the phrase “part of” used to describe the relationship?
 - A door is “part of” a car
 - Are some operations on the whole automatically applied to its parts?
 - Move the car, move the door.
 - Are some attribute values propagated from the whole to all or some of its parts?
 - The car is blue, therefore the door is blue.
 - Is there an intrinsic asymmetry to the relationship where one class is subordinate to the other?
 - A door **is** part of a car. A car **is not** part of a door.

Composition

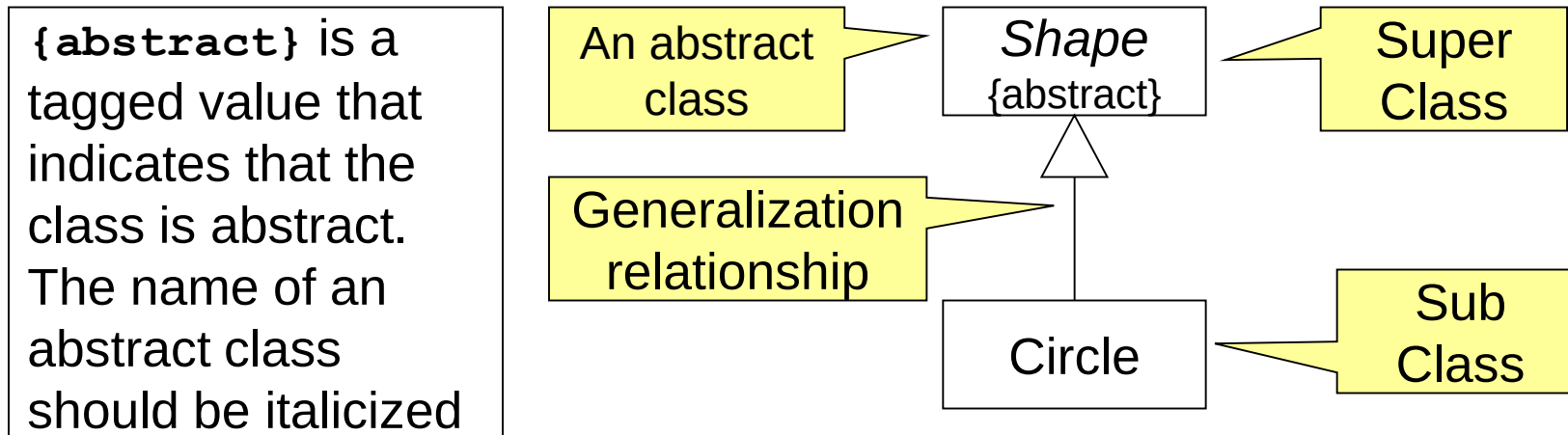
- A strong form of aggregation
 - The whole is the sole owner of its part.
 - The part object may belong to only one whole
 - Multiplicity on the whole side must be zero or one.
 - The life time of the part is dependent upon the whole.
 - The composite must manage the creation and destruction of its parts.



UML Class Diagrams

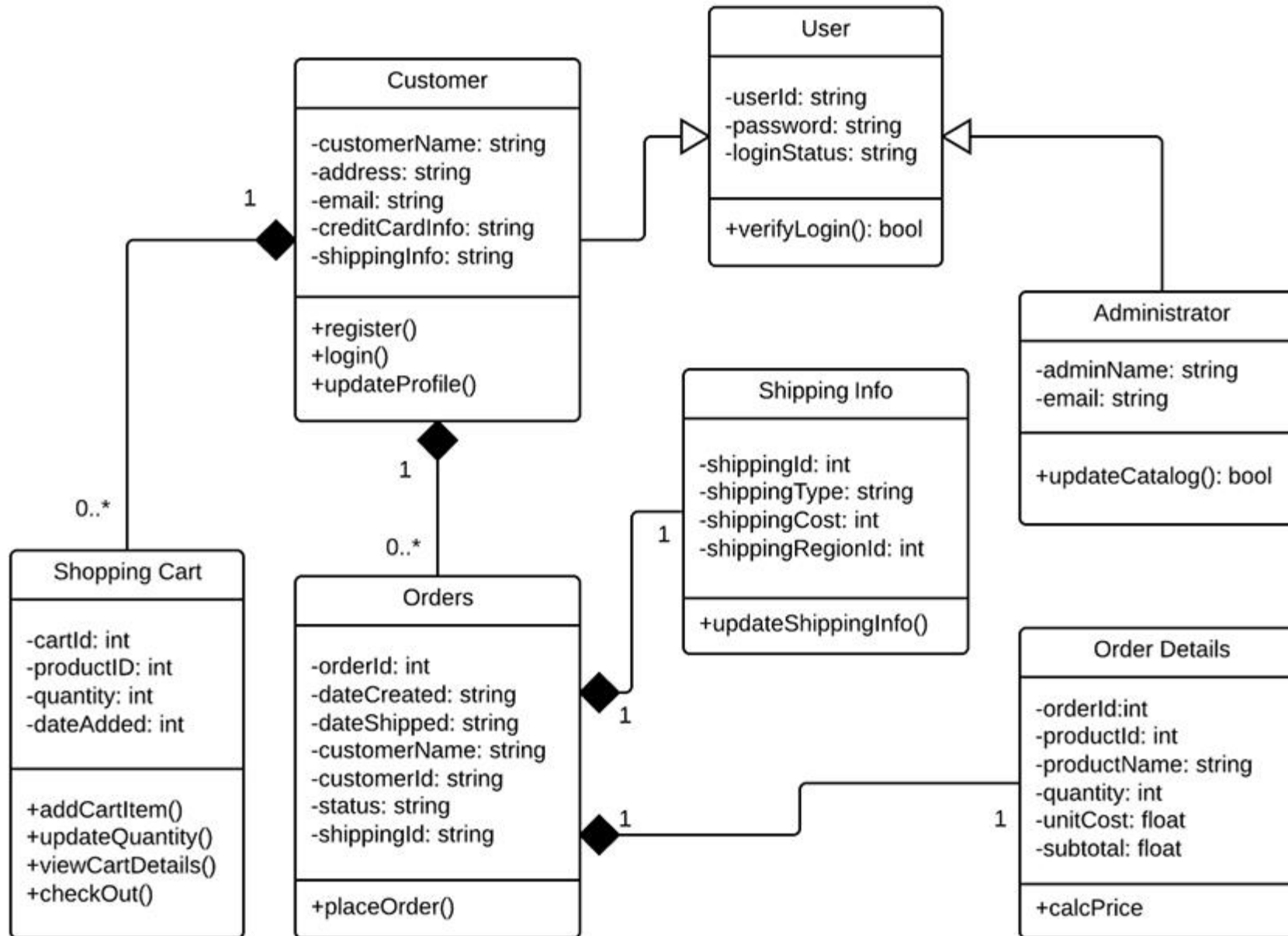
Generalization

- Indicates that objects of the specialized class (subclass) are substitutable for objects of the generalized class (super-class).
 - “is kind of” relationship.

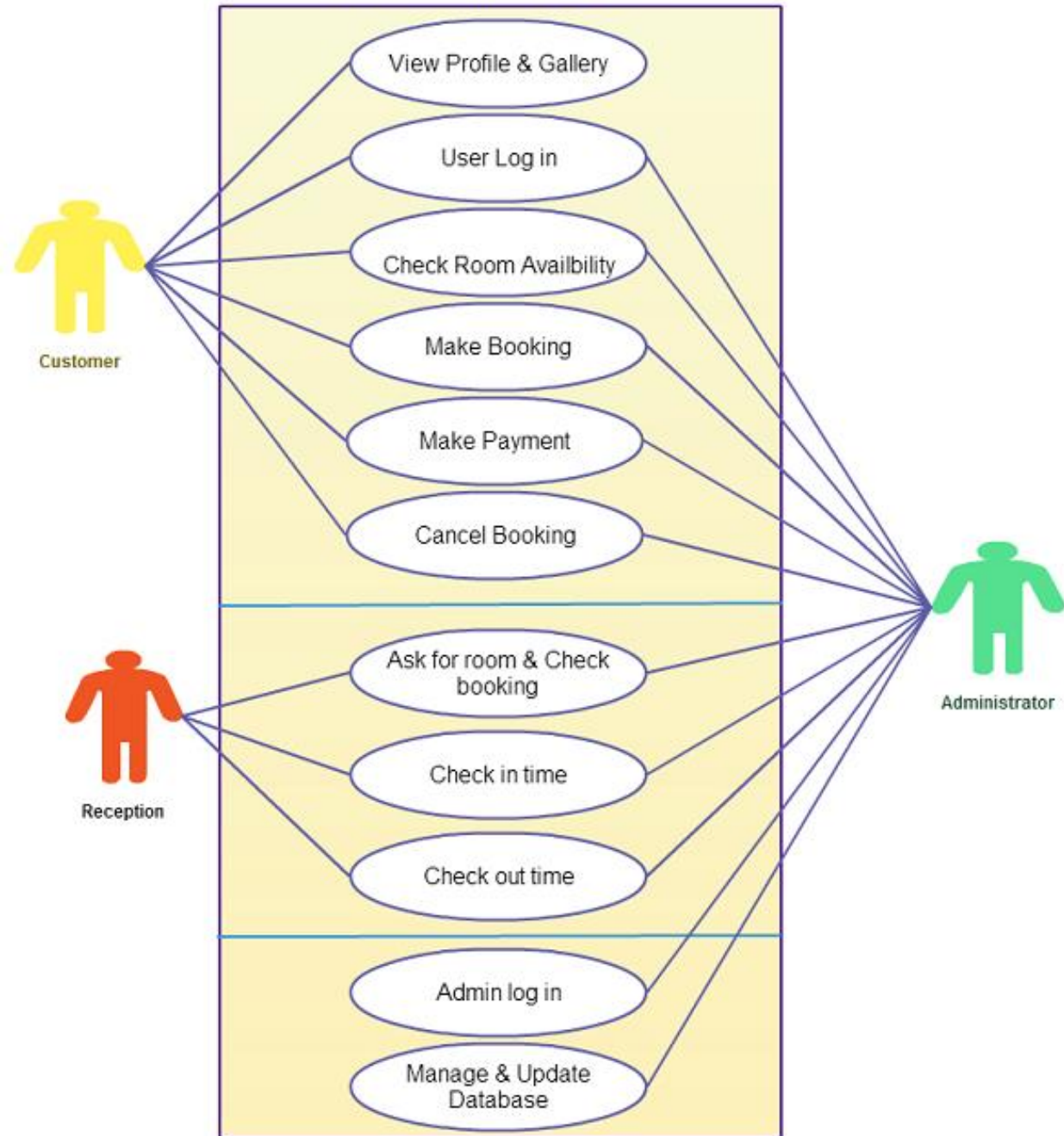


Generalization

- A sub-class inherits from its super-class
 - Attributes
 - Operations
 - Relationships
- A sub-class may
 - Add attributes and operations
 - Add relationships
 - Refine (override) inherited operations
- A generalization relationship **may not** be used to model interface implementation.



Hotel Management System



Summary

- What is a class diagram?
- Identifying classes/Operations/Attributes
- Associations
 - Simple associations
 - Multiplicity
 - Aggregation
 - Composition
 - Generalization